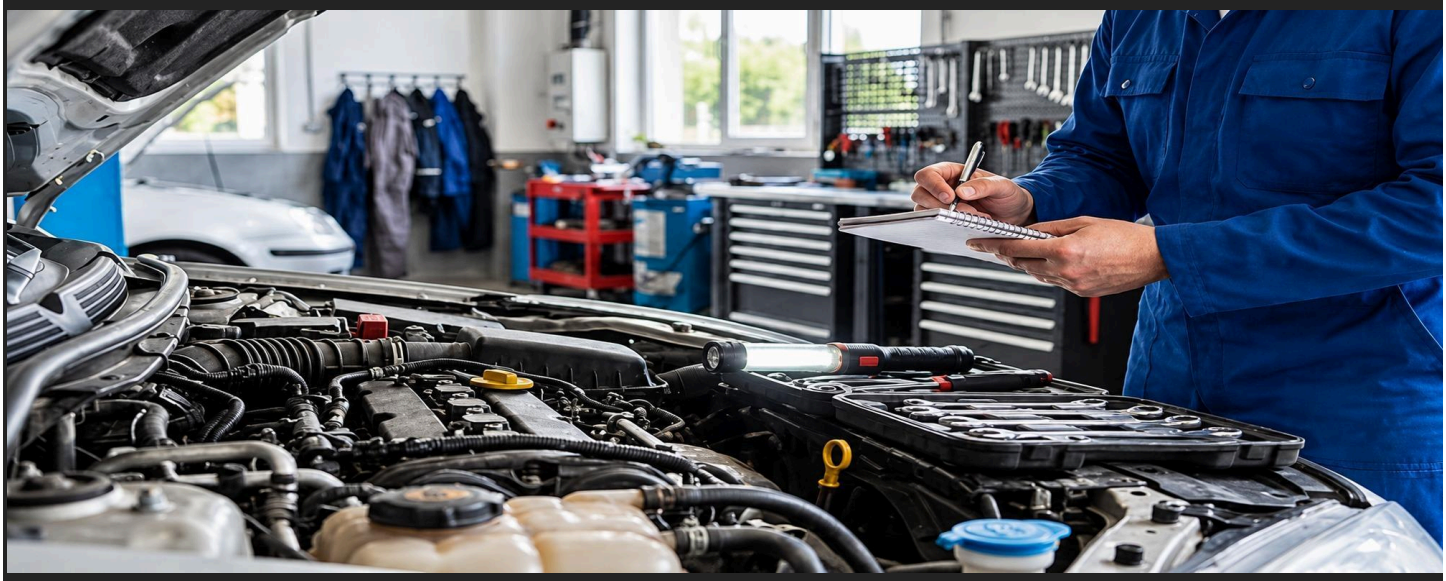
UNIVERSIDAD MAYOR
para espíritus emprendedores

INFORME DE PROYECTO UNIVERSITARIO · HITO 2 · INFORME INICIAL

Repuestos Exactos

Arquitectura de desarrollo móvil y web para el diagnóstico automotriz OBD-II y la cotización optimizada de repuestos mediante recomendación analítica.

Diagnóstico, cotización y reporte para talleres y conductores.



UNIVERSIDAD MAYOR

Facultad de Ciencias, Ingeniería y Tecnología · Escuela de Ingeniería Civil · Ingeniería Civil en Computación e Informática

DATOS DEL TRABAJO

Asignatura y código: Arquitectura de desarrollo (Móvil y Web) · INGT1003-001

Equipo: Repuestos Exactos

Integrantes: Javier Chandía, Matias Daza, Victoria Escobar, Adrián Perugini

Docente: Carlos Muñoz S.

Ciudad: Santiago / Rancagua

Fecha: 28/09/2026

Índice

1 Introducción 1
2 Trazabilidad con el Hito 1 1
3 Diseño de la API 2
4 Separación de responsabilidades y capas 5
5 Persistencia y manejo de credenciales 7
6 Decisión de arquitectura: monolito o distribuido 9
7 Limitaciones y trabajos pendientes 10
8 Conclusión 11
9 Referencias 11
10 Anexos 12
Anexo A. Declaración de uso de inteligencia artificial 12
Anexo B. Repositorio en GitHub 12

1 Introducción

Repuestos Exactos es una aplicación móvil y web que conecta el diagnóstico OBD-II de un vehículo con la cotización de sus repuestos y recomienda la compra según costo, durabilidad y compatibilidad (Chandía et al., 2026). El Hito 1 presentó el problema, la arquitectura por capas y el prototipo del front end. Este documento describe el back end que sostiene esa propuesta: una API REST escrita en Dart con Dart Frog (Very Good Ventures, s. f.-a), conectada a una base de datos PostgreSQL 16 (PostgreSQL Global Development Group, s. f.-b).

El documento sigue el orden de la pauta del hito: los cambios respecto del Hito 1, el diseño de la API, la separación de capas, la persistencia y el manejo de credenciales. El alcance real del back end es el siguiente: autenticación, configuración del taller, historial de sesiones de diagnóstico, catálogo de marcas, modelos y códigos de falla (DTC) con sus repuestos, y búsqueda de un vehículo registrado por VIN. La creación real de diagnósticos y el cotizador de tiendas externas siguen pendientes y se declaran como tales en la sección de limitaciones.

2 Trazabilidad con el Hito 1

El Hito 1 dejó varias decisiones abiertas: la forma de obtener precios, el proveedor de la nube, la confirmación de PostgreSQL y la estrategia de sincronización con SQLite (Chandía et al., 2026). La Tabla 1 resume qué se movió, por qué y qué impacto tuvo.

Aspecto	Hito 1	Hito 2	Motivo e impacto
Datos del front end	Repositorios simulados en lib/data/	Inicio de sesión, historial, búsqueda y ajustes consumen la API con el cliente dio (Pub.dev, s. f.-a)	Se pasa de datos de prueba a un flujo real. El front debe manejar errores de red y de autenticación. La captura de diagnósticos conserva su alcance de prototipo.
Servidor	«Servicios en la nube»; API propia o consulta directa a terceros, por definir	API REST propia con Dart Frog, ejecutada en local en el puerto 8081	Se mantiene Dart, el lenguaje elegido en el Hito 1. El proveedor de nube y su costo siguen sin definirse.

Base central	PostgreSQL «ya en uso»; confirmación pendiente	PostgreSQL 16 en contenedor Docker, esquema versionado en migraciones V000 a V010	Queda confirmado el motor. El aislamiento entre talleres pasa a la base de datos mediante seguridad a nivel de fila, y no solo a la aplicación.
Identidad del usuario	Saludo fijo en el prototipo	Token Bearer opaco, de una hora, con roles por usuario	Cada taller ve solo sus datos. Reiniciar la API cierra todas las sesiones.
Identificación del vehículo	Lectura de VIN por OBD-II, diseñada y no implementada	Búsqueda de un vehículo registrado por VIN en GET /search	Es una búsqueda local en la base. La lectura del VIN desde el escáner sigue pendiente.
Cotización	Cotizador que consulta tiendas en línea	La base guarda la cotización con copia (snapshot) del precio y del margen; el cotizador externo no está implementado	Se asegura la trazabilidad de la cotización. Falta la fuente externa de precios.
Persistencia local (SQLite)	Propuesta, con sincronización por definir	[POR COMPLETAR: estado de la sincronización en este hito]	Estrategia de sincronización local en evaluación continua.

Nota. Los motivos reflejan la documentación de los repositorios y deben ser confirmados por el equipo. Elaboración propia a partir de Chandía et al. (2026).

3 Diseño de la API

3.1 Convenciones

La API intercambia JSON y usa HTTP según su semántica estándar (Fielding et al., 2022). Solo son públicas las rutas /, /health/database y /auth/login; toda ruta nueva queda

protegida por defecto. Las rutas se declaran por estructura de carpetas, y Dart Frog genera el enrutador a partir de ellas (Very Good Ventures, s. f.-d).

3.2 Recursos

La Tabla 2 lista los recursos expuestos. Los códigos de estado que el repositorio no documenta figuran como «Por confirmar» y deben completarse con la colección de Postman o con el código de cada ruta.

Verbo	Ruta	Acceso	Parámetros	Éxito	Errores
GET	/	Público	Ninguno	200	Por confirmar
GET	/health/database	Público	Ninguno	200	503, 405
POST	/auth/login	Público	Cuerpo JSON: email, password	Por confirmar	Por confirmar
GET	/auth/me	Bearer	Ninguno	Por confirmar	Por confirmar
POST	/auth/logout	Bearer	Ninguno	Por confirmar	Por confirmar
GET	/settings	Bearer; solo su taller	Ninguno	Por confirmar	Por confirmar
GET	/sessions	Bearer	Filtro por estado; cursor de paginación	Por confirmar	Por confirmar
GET	/search	Bearer; administración y mecánica	vin (consulta)	Por confirmar	Por confirmar
GET	/catalog/brands	Bearer	Ninguno	Por confirmar	Por confirmar
GET	/catalog/brands/{brand}/models	Bearer	brand (ruta)	Por confirmar	Por confirmar
GET	/catalog/dtc	Bearer	Ninguno	Por confirmar	Por confirmar

GET	/catalog/dtc/{code}/parts	Bearer (ruta)	Por confirmar	Por confirmar

Nota. Elaboración propia a partir del README del back end y del enrutador generado por Dart Frog (server.dart).

3.3 Ejemplos de carga útil

Solicitud de inicio de sesión (POST /auth/login), con un usuario de las semillas de demostración:

POST /auth/login

```
{
  "email": "admin@losaromos.example",
  "password": "CAMBIAR_clave_demo"
}
```

Respuesta: [POR COMPLETAR: cuerpo real devuelto por la API, con el token].

Respuesta de GET /health/database con la conexión correcta (HTTP 200):

GET /health/database (Éxito 200)

```
{"status":"ok","database":"connected"}
```

Respuesta cuando falla la conexión, faltan las tablas comprobadas o el rol no coincide (HTTP 503):

GET /health/database (Error 503)

```
{"status":"error","database":"unavailable"}
```

La API no devuelve excepciones SQL ni valores de configuración en sus errores.

3.4 Códigos de estado

La Tabla 3 contrastar el significado de cada código (Fielding et al., 2022) con lo que el repositorio documental hoy.

Código	Significado	Documentado en el repositorio
200	Solicitud exitosa	Sí (/health/database)
401	Falta autenticación o el token no es válido	No

403	Autenticado, pero sin permiso para el recurso	No
404	Recurso inexistente	No
405	Método no admitido	Sí (/health/database)
503	Servicio no disponible (falla la base)	Sí (/health/database)

Nota. Los códigos «No» deben verificarse en el código antes de la entrega.

3.5 Autenticación y acceso desde el navegador

El cliente envía el encabezado Authorization: Bearer <token> (Jones y Hardt, 2012). El token es opaco, dura una hora y su identidad se guarda en el servidor. En cada petición protegida se revalidan el usuario, el taller activo y el rol.

Para el front web, la API acepta por defecto los orígenes http://localhost:8080 y http://127.0.0.1:8080. La variable CORS_ALLOWED_ORIGINS define una lista de orígenes exactos, sin comodines. Las solicitudes de verificación previa (preflight) OPTIONS se resuelven antes de la autenticación (MDN Web Docs, s. f.).

3.6 Criterio de versionado

La API no usa hoy un prefijo de versión. [CONFIRMAR CON EL EQUIPO] Se propone el siguiente criterio: la API actual se considera la versión 1 implícita. Un cambio incompatible (eliminar un campo, cambiar su tipo o cambiar el significado de una ruta) se publica bajo un prefijo nuevo, /v2, y la versión anterior se mantiene durante un período de transición. Un cambio compatible (agregar un campo opcional o una ruta nueva) no cambia la versión. Con el enrutamiento por carpetas de Dart Frog, esto equivale a crear una carpeta routes/v2/ junto a las rutas existentes (Very Good Ventures, s. f.-d).

4 Separación de responsabilidades y capas

4.1 Estructura de carpetas

La Tabla 4 asigna cada archivo del repositorio Backend_Repuestos_Exactos a su capa. La base de datos vive en un repositorio aparte, RE-db.

Archivo o carpeta	Capa	Responsabilidad

main.dart	Arranque y composición	init crea la base de datos, el almacén de sesiones y la lista de orígenes CORS. run los inyecta con provider y aplica el middleware de CORS (Very Good Ventures, s. f.-b; s. f.-c).
lib/config/database_config.dart	Configuración	Lee y valida las variables de entorno: presencia de valores y rango del puerto.
lib/database/database.dart	Acceso a datos	Pool de PostgreSQL con máximo de 5 conexiones y 5 segundos de espera. Ejecuta las consultas.
lib/auth/session_store.dart	Sesión	Almacén de sesiones del servidor.
lib/http/cors_middleware.dart	Transversal HTTP	Política de orígenes permitidos.
routes/_middleware.dart	Transversal HTTP	Middleware raíz, aplicado a todas las rutas. Según el README, protege por defecto las rutas no públicas.
routes/sessions/_middleware.dart, routes/catalog/_middleware.dart	Transversal HTTP	[POR COMPLETAR: función de cada middleware de carpeta]
routes/**/*.dart	Controladores HTTP	Reciben la petición, leen las dependencias del contexto y responden. Son 12 rutas.
tool/check_database.dart	Herramienta	Diagnóstico local de la conexión y listado de tablas.
test/	Pruebas	Pruebas unitarias y de integración.
RE-db (otro repositorio)	Datos	Migraciones, roles, seguridad por fila y las 43 sentencias preparadas que usa el back end.

Nota. Elaboración propia a partir del README, de main.dart y de server.dart. [VERIFICAR: que los controladores de routes/ no contengan SQL ni reglas de negocio].

4.2 Recorrido de una petición

El siguiente recorrido sigue la ruta GET /sessions, desde el cliente hasta la base de datos:

1. El front Flutter envía la solicitud con dio y el encabezado Authorization: Bearer <token>.
2. El middleware de CORS valida el origen. Si es una solicitud OPTIONS, la responde antes de la autenticación.
3. El middleware raíz identifica el token en el almacén de sesiones y revalida usuario, taller activo y rol. Sin sesión válida, la petición se corta aquí.
4. El middleware de la carpeta sessions/ se ejecuta antes del controlador (Very Good Ventures, s. f.-c).
5. El controlador routes/sessions/index.dart obtiene Database del contexto, mediante la inyección de dependencias de Dart Frog (Very Good Ventures, s. f.-b), y pide el historial con los filtros y el cursor recibidos.
6. Database abre una transacción y asume el rol del usuario con SET LOCAL ROLE (PostgreSQL Global Development Group, s. f.-d). Luego fija el taller con set_config(..., true), de modo que ambos valores solo duran lo que dura la transacción.
7. PostgreSQL ejecuta la sentencia preparada. La seguridad a nivel de fila filtra los registros al taller del usuario (PostgreSQL Global Development Group, s. f.-c).
8. La respuesta vuelve como JSON paginado por la misma cadena, en sentido inverso.

Nota. Los pasos 4 a 6 describen el diseño documentado en los repositorios. [VERIFICAR contra el código de routes/sessions/index.dart y lib/database/database.dart].

5 Persistencia y manejo de credenciales

5.1 Modelo de datos

La base de datos es un repositorio independiente (RE-db), con migraciones versionadas de V000 a V010: roles, tipos, tablas, comentarios, funciones y disparadores, índices, privilegios, seguridad por fila y dos ajustes posteriores. El diccionario de datos completo, con claves, relaciones y tipos de cada columna, se genera desde los comentarios de la base y está en docs/modelo/05_ diccionario datos.md de ese repositorio. La Tabla 5 resume las decisiones de diseño.

Decisión	Implementación
Varios talleres en una misma base	Cada tabla operativa lleva taller_id. Las claves foráneas compuestas (padre_id, taller_id) impiden referencias entre talleres. La seguridad por fila se aplica con FORCE (PostgreSQL Global Development Group, s. f.-c).
Catálogo global, operación local	Marcas, modelos, DTC, componentes y repuestos son comunes y de solo lectura. Cada taller aporta su margen.
Backend sin privilegios propios	app_backend_user no lee nada hasta asumir el rol del usuario dentro de la transacción.
Dinero inmutable	Precios en BIGINT. La cotización guarda una copia del precio y del margen, y su total queda congelado al cerrarse.
Privacidad por diseño	Sin RUT, anonimización irreversible y auditoría sin datos personales.

Nota. Elaboración propia a partir del README de RE-db. Tablas comprobadas por la API: vehículo, sesión de diagnóstico y cotización.

5.2 Script de creación reproducible

La base se levanta con Docker Compose (Docker Inc., s. f.) y se reconstruye desde cero con cuatro comandos. El criterio de aceptación es que la suite termine con la línea test: to PASS (518 pruebas en 12 archivos).

Comandos de despliegue y pruebas

```
cp .env.example .env
docker compose up -d --wait
bash db/scripts/reset.sh && bash db/scripts/migrate.sh \
  && bash db/scripts/seed.sh && bash db/tests/run_all.sh
```

La base queda publicada solo en 127.0.0.1:5433, y toda conexión exige clave con autenticación SCRAM, también dentro del contenedor (PostgreSQL Global Development Group, s. f.-a). La API se inicia después con dart_frog dev --port 8081.

5.3 Credenciales

Ninguna clave está escrita en el código. El docker-compose.yml lee todo desde variables de entorno, y cada repositorio entrega un archivo .env.example con marcadores de posición. El archivo .env real no se versiona. La API se conecta como app_backend_user y no como

superusuario del contenedor. Las variables dotenv se cargan con el paquete dotenv (Pub.dev, s. f.-b), y las variables del proceso tienen prioridad sobre el archivo.

Repositorio	Variables	Uso
Back end	DB_HOST, DB_PORT, DB_NAME, DB_USER, DB_PASSWORD, DB_SSL_MODE, CORS_ALLOWED_ORIGINS	Conexión de la API y orígenes permitidos. TLS usa verify-full por defecto; el ejemplo local usa disable.
Base de datos	POSTGRES_USER, POSTGRES_PASSWORD, POSTGRES_DB, APP_BACKEND_PASSWORD, CLAVE_DEMO	Creación del contenedor, clave del rol de la API y clave común de los usuarios de demostración.

Nota. Elaboración propia a partir de los archivos README y docker-compose.yml. [VERIFICAR: que .env figure en el .gitignore de los tres repositorios y que nunca se haya subido al historial de Git].

6 Decisión de arquitectura: monolito o distribuido

El equipo adopta un monolito modular: una sola API en Dart Frog, un solo proceso, conectada a una única base PostgreSQL. La decisión se sostiene en cuatro criterios.

Criterio	Aplicación al proyecto
Tamaño del equipo	Cuatro integrantes (Chandía et al., 2026). Cada servicio adicional sumaría su propio código, despliegue y mantención.

Carga esperada	Uso de oficina: cada mecánico consulta catálogo, historial y búsquedas puntuales dentro de su taller. [COMPLETAR: número estimado de talleres, usuarios y peticiones por minuto]. El pool de conexiones, de 5 como máximo, es suficiente para ese perfil.
Costo de operación	Un proceso de API y un contenedor de base de datos. El proveedor de nube sigue sin definirse, por lo que menos piezas significan menos costo.
Complejidad de despliegue	La base se levanta con Docker Compose (Docker Inc., s. f.) y la API con un comando. Una arquitectura distribuida exigiría orquestación, descubrimiento de servicios y trazabilidad entre servicios.

Nota. Elaboración propia.

Un argumento adicional es la consistencia de los datos. La cotización guarda una copia del precio y del margen, y su total queda congelado al cerrarse. Esas operaciones deben ser atómicas dentro de una transacción de PostgreSQL. Si se repartieran en servicios distintos, requerirían transacciones distribuidas, un costo que el proyecto no justifica.

El monolito conserva fronteras internas: las rutas se organizan en carpetas por recurso (auth, catalog, sessions, search, settings), y Dart Frog genera el enrutador a partir de esa estructura (Very Good Ventures, s. f.-d). Esa separación facilita extraer un módulo más adelante.

Costo asumido. Toda la API se despliega y escala como un bloque, y una falla grave en un módulo puede afectar a los demás. Además, los módulos comparten versión y ciclo de publicación.

Cuándo se revisará. El primer candidato a separarse es el cotizador externo, porque depende de cuotas y tiempos de respuesta de terceros. También se revisará si la carga sostenida supera la capacidad de una instancia [COMPLETAR: umbral] o si el proyecto pasa a tener equipos que se desplieguen de forma independiente.

7 Limitaciones y trabajos pendientes

• La creación real de diagnósticos sigue pendiente. La captura de diagnósticos del front conserva su alcance de prototipo.

• El cotizador de tiendas externas no está implementado. La base ya guarda la cotización con trazabilidad.

• La contraseña del usuario llega en claro al motor, que la procesa con crypt(). En producción, el hash debe calcularlo el backend.

• La base no usa TLS, PgBouncer ni recuperación a un punto en el tiempo, y los respaldos no están cifrados.

• Las sesiones se guardan en memoria. Reiniciar la API cierra todas.

• Cuatro reglas quedan a cargo de la API y de la configuración de producción, porque la base no las hace cumplir.

• El proveedor de nube, el plan y su costo mensual siguen por definir.

• La API no tiene versión explícita (ver el criterio propuesto).

8 Conclusión

El back end convirtió el prototipo del Hito 1 en un sistema con autenticación real, datos por taller y una base reproducible. La principal decisión técnica fue delegar el aislamiento entre talleres a PostgreSQL mediante seguridad por fila, de modo que un error de la aplicación no pueda mezclar datos entre talleres. Quedan pendientes el cotizador externo, la creación de diagnósticos y el despliegue en la nube, que son los siguientes pasos del proyecto.

9 Referencias

- Docker Inc. (s. f.). Docker Compose overview. https://docs.docker.com/compose/
- Fielding, R., Nottingham, M., y Reschke, J. (2022). HTTP semantics (RFC 9110). Internet Engineering Task Force. https://www.rfc-editor.org/rfc/rfc9110
- Jones, M., y Hardt, D. (2012). The OAuth 2.0 authorization framework: Bearer token usage (RFC 6750). Internet Engineering Task Force. https://www.rfc-editor.org/rfc/rfc6750
- MDN Web Docs. (s. f.). Cross-Origin Resource Sharing (CORS). Mozilla. https://developer.mozilla.org/en-US/docs/Web/HTTP/CORS
- PostgreSQL Global Development Group. (s. f.-a). Password authentication. En PostgreSQL 16 documentation. https://www.postgresql.org/docs/16/auth-password.html
- PostgreSQL Global Development Group. (s. f.-b). PostgreSQL 16 documentation. https://www.postgresql.org/docs/16/
- PostgreSQL Global Development Group. (s. f.-c). Row security policies. En PostgreSQL 16 documentation. https://www.postgresql.org/docs/16/ddl-rowsecurity.html
- PostgreSQL Global Development Group. (s. f.-d). SET ROLE. En PostgreSQL 16 documentation. https://www.postgresql.org/docs/16/sql-set-role.html
- Pub.dev. (s. f.-a). dio (Versión 5.x) [Paquete de software]. https://pub.dev/packages/dio
- Pub.dev. (s. f.-b). dotenv (Versión 4.2.0) [Paquete de software]. https://pub.dev/packages/dotenv
- Pub.dev. (s. f.-c). postgres (Versión 3.5.20) [Paquete de software]. https://pub.dev/packages/postgres

- Very Good Ventures. (s. f.-a). Dart Frog documentation. https://dartfrog.vgv.dev/docs/overview
- Very Good Ventures. (s. f.-b). Dependency injection. En Dart Frog documentation. https://dartfrog.vgv.dev/docs/basics/dependency-injection
- Very Good Ventures. (s. f.-c). Middleware. En Dart Frog documentation. https://dartfrog.vgv.dev/docs/basics/middleware
- Very Good Ventures. (s. f.-d). Routes. En Dart Frog documentation. https://dartfrog.vgv.dev/docs/basics/routes

10 Anexos

Anexo A. Declaración de uso de inteligencia artificial

De acuerdo con la pauta de la asignatura, el equipo declara las herramientas de inteligencia artificial utilizadas en el Hito 1, la parte del trabajo en que se emplearon y el modo en que se verificaron los resultados.

- **Gemini (Google), versión de pago:** búsqueda, rastreo y consolidación de fuentes durante la investigación, y asistencia en la codificación del prototipo.
- **Claude (Anthropic):** redacción y organización del informe, preparación de figuras, revisión del cumplimiento de la pauta y auditoría de calidad del documento.
- **Entorno de desarrollo del prototipo:** asistencia en la escritura de código, con las herramientas de IA integradas.

Verificación. Cada cita y referencia se contrastó con su fuente original, en la medida en que el sitio fue accesible. Las cifras de precios se calcularon a partir del registro del Anexo C. El código generado con asistencia se revisó antes de incorporarlo al repositorio, y los integrantes pueden explicar sus partes.

Responsabilidad. Los integrantes revisaron el contenido incorporado y mantienen la responsabilidad sobre las decisiones del proyecto, la arquitectura, la selección tecnológica y la información presentada.

Anexo B. Repositorio en GitHub

El código fuente, la estructura de carpetas en Flutter, los módulos de lectura BLE y la integración de servicios están disponibles para auditoría técnica en el repositorio público https://github.com/Risu-31/Repuestos-Exactos.